Introduction to Generative Models

CSC 5666: Advanced Machine Learning, Fall 2026
Professor Austin P. Wright: Course Notes

Abstract
Overview of generative models, model types, tasks, and evaluation. See textbook Chapter 20.

Contents

I Types of Generative Model 1
II Goals of Generative Modeling 3
III Evaluating Generative Models 7

Textbook: §§20.1–20.2. Stanford CS236, Introduction.

I Types of Generative Model

A **generative model** is a joint probability distribution $p(x)$, for $x \in \mathcal{X}$. In some cases, the model may be conditioned on inputs or covariates $c \in \mathcal{C}$, which gives rise to a **conditional generative model** of the form $p(x|c)$.

Fitting one means picking the parameters within a family of model distributions that minimizes some notion of distance to the data distribution,

$$\min_{\theta \in \mathcal{M}} d(p_{\text{data}}, p_\theta), \tag{1}$$

where p_{data} is accessed only through the dataset and d is a notion of distance between probability distributions. Discriminative models predict a label for any given datapoint. Generative models, on the other hand, learn a joint distribution over the entire data.

At a high level, we can distinguish between **deep generative models** (DGM) which use deep neural networks to learn a complex mapping from a single latent vector z to the observed data x, and more "classical" **probabilistic graphical models** (PGM), that map a set of interconnected latent variables $z_1, ..., z_L$ to the observed variables $x_1, ..., x_D$ using simpler, often linear, mappings [1]. The main kinds of DGM are: *variational autoencoders* (VAE), *autoregressive models* (ARM), *normalizing flows*, *diffusion models*, *energy based models* (EBM), and *generative adversarial networks* (GAN), many of which we will cover in more detail in the coming lectures.

[1] Though in fact these classes afford plenty of combination, such as using neural networks to map relations within a PGM as well as enforcing state space structure within a DGM

We can categorize these models in terms of the following criteria:

- *Density:* does the model support pointwise evaluation of the probability density function $p(x)$, and if so, is this fast or slow, exact, approximate or a bound, etc? For implicit models, such as GANs, there is no well-defined density $p(x)$. For other models, we can only compute a lower bound on the density (VAEs), or an approximation to the density (EBMs, UPGMs).
- *Sampling:* does the model support generating new samples, $x \sim p(x)$, and if so, is this fast or slow, exact or approximate? Directed PGMs, VAEs, and GANs all support fast sampling. However, undirected PGMs, EBMs, ARM, diffusion, and flows are slow for sampling.
- *Training:* what kind of method is used for parameter estimation? For some models (such as AR, flows and directed PGMs), we can perform exact

maximum likelihood estimation (MLE), although the objective is usually non-convex, so we can only reach a local optimum. For VAEs we maximize a lower bound on the likelihood; for EBMs and UGMs, an approximation to it. For GANs we have to use min-max training, which can be unstable, with no clear objective function to monitor.

- *Latents:* does the model use a latent vector z to generate x or not, and if so, is it the same size as x or is it a potentially compressed representation? For example, ARMs do not use latents; flows and diffusion use latents, but they are not compressed. Graphical models, including EBMs, may or may not use latents.
- *Architecture:* what kind of neural network should we use, and are there restrictions? Flows are restricted to invertible networks with tractable Jacobians; EBMs can use any model we like.

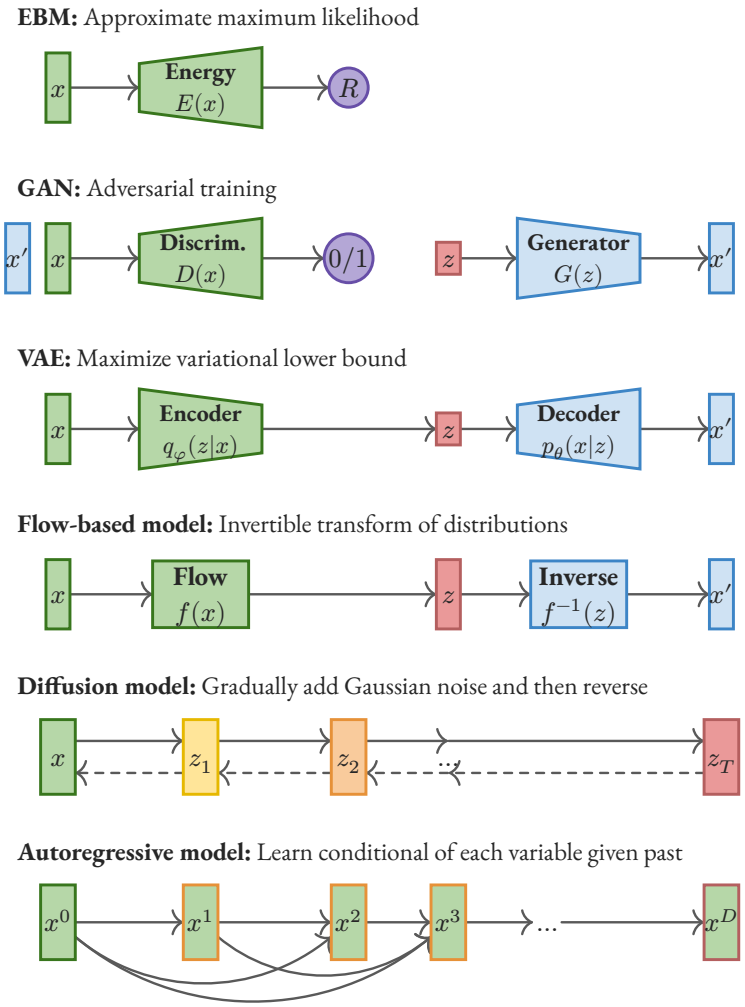


Figure 1: Summary of various kinds of deep generative models. Here x is the observed data, z is the latent code, and x' is a sample from the model. AR models do not have a latent code z; for diffusion and flow models, z is the same size as x.

Adapted from Figure 1 of Weng (2021) and Fig. 20.1 of Murphy (2023).

II Goals of Generative Modeling

Textbook: §20.3

There are several different kinds of tasks that we can use generative models for, as we discuss below.

> ***Discussion***
> When considering the a *conditional generative model*, $p(x|c)$, where the goal is to genrate a distribution over data x given some fixed *condition*, notice the similarity to *supervised learning*, where the goal is the predict a class given input data, i.e. $p(c|x)$.
>
> Describe a familiar supervised learning problem that you know well. Then articulate what a corresponding conditional generative modeling problem for the same domain would look like. Finally, sketch out some different analysis or use case scenarios where one or the other modeling problem may be more useful. Which problems are "harder"? Why?

Generating Data

One of the main goals of generative models is to generate (create) new data samples. This is sometimes called generative AI. For example, if we fit a model $p(x)$ to images of faces, we can sample new faces from it.

To control what is generated, it is useful to use a conditional generative model of the form $p(x|c)$. Here are some examples:

- c = text prompt, x = image. This is a text-to-image model (e.g., "a cute corgi lives in a house made out of sushi").
- c = image, x = image. This is an image-to-image model, and can be used for image colorization, inpainting, uncropping, JPEG artefact restoration, etc.
- c = sequence of English words, x = sequence of French words. This is a sequence-to-sequence model, which is useful for machine translation.

Note that, in the conditional case, we sometimes denote the inputs by x and the outputs by y. In this case the model has the familiar form $p(y|x)$. In the special case that y denotes a low dimensional quantity, such as a integer class label, $y \in \{1, ..., C\}$, we get a predictive (discriminative) model. The main difference between a discriminative model and a conditional generative model is this: in a discriminative model, we assume there is one correct output, whereas in a conditional generative model, we assume there may be multiple correct outputs. This makes it harder to evaluate generative models, as we discuss in the next section.

Density Estimation

The task of **density estimation** refers to evaluating the probability of an observed data vector, i.e., computing $p(x)$. This can be useful for outlier detection, generative classifiers, model comparison, etc.

A simple approach to this problem, which works in low dimensions, is to use **kernel density estimation** or KDE, which has the form

$$p(x|\mathcal{D}) = \frac{1}{N}\sum_{n=1}^{N} \mathcal{K}_h(x - x_n). \tag{2}$$

Here $\mathcal{D} = \{x_1, ..., x_N\}$ is the data, and $\mathcal{K}_h$ is a density kernel with **bandwidth** h, which is a function $\mathcal{K} : \mathbb{R} \to \mathbb{R}_+$ such that $\int \mathcal{K}(x)\,\mathrm{d}x = 1$ and $\int x\mathcal{K}(x)\,\mathrm{d}x = 0$. In higher dimensions, KDE suffers from the curse of dimensionality, and we need to use parametric density models $p_\theta(x)$ of some kind.

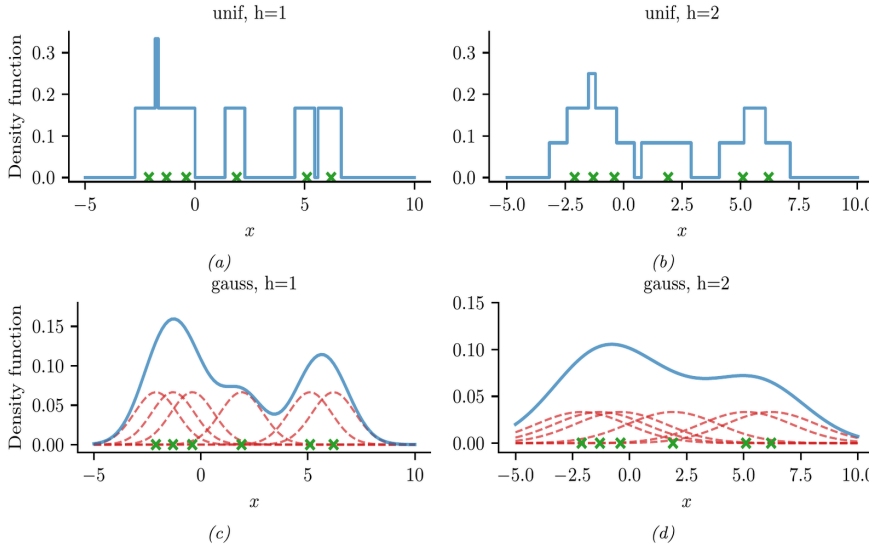


Figure 2: A nonparametric (Parzen) density estimator in 1d estimated from 6 datapoints, denoted by x. Top row: uniform kernel. Bottom row: Gaussian kernel. Left column: bandwidth $h = 1$. Right column: $h = 2$. Murphy (2023), Fig. 20.5.

Imputation

The task of **imputation** refers to "filling in" missing values of a data vector or data matrix. For example, suppose $\boldsymbol{X}$ is an $N \times D$ matrix of data (think of a spreadsheet) in which some entries, call them $\boldsymbol{X}_m$, may be missing, while the rest, $\boldsymbol{X}_o$, are observed. A simple way to fill in the missing data is to use the mean value of each feature, $\mathbb{E}[x_d]$; this is called **mean value imputation**. However, this ignores dependencies between the variables within each row, and does not return any measure of uncertainty. We can generalize this by fitting a generative model to the observed data, $p(\boldsymbol{X}_o)$, and then computing samples from $p(\boldsymbol{X}_m|\boldsymbol{X}_o)$. This is called **multiple imputation**.

A	B	C
6	6	NA
NA	6	0
NA	6	NA
10	10	10
10	10	10
10	10	10
9	8	7.5

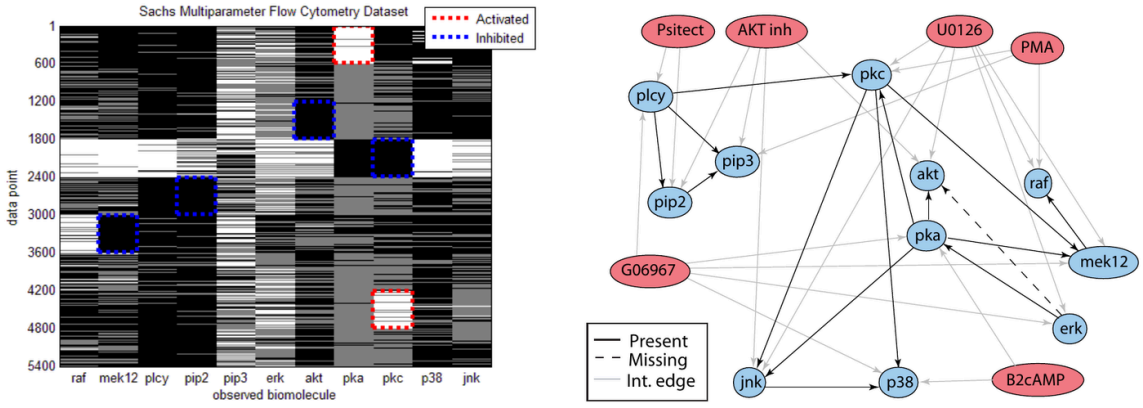
A	B	C
6	6	7.5
9	6	0
9	6	7.5
10	10	10
10	10	10
10	10	10
9	8	7.5

Table 1: Missing data imputation. Left: input data: NA means "not available" (missing), and the bottom row shows the mean of each column. Right: output data, where NA values are replaced by the mean. Redrawn from Murphy (2023), Fig. 20.6.

Structure Discovery

Some kinds of generative models have latent variables z, which are assumed to be the "causes" that generated the observed data x. We can use Bayes' rule to invert the model to compute $p(z|x) \propto p(z)p(x|z)$. This can be useful for discovering latent, low-dimensional patterns in the data.

For example, suppose we perturb various proteins in a cell and measure the resulting phosphorylation state using a technique known as flow cytometry. Each row represents a data sample $x_n \sim p(\cdot\,|a_n, z)$, where $x \in \mathbb{R}^{11}$ is a vector of outputs (phosphorylations), $a \in \{0,1\}^6$ is a vector of input actions (perturbations) and z is the unknown cellular signaling network structure. We can infer the graph structure $p(z|\mathcal{D})$ using graphical model structure learning techniques.



(a) *(b)*

Figure 3: (a) A design matrix of 5400 datapoints (rows) measuring the state of 11 proteins (columns) under different experimental conditions, discretized into low (black), medium (grey), and high (white). (b) A directed graphical model of dependencies between proteins (blue circles) and experimental interventions (pink ovals), inferred from this data. We plot all edges for which $p(G_{ij} = 1|\mathcal{D}) > 0.5$. Dotted edges were not discovered by the algorithm; solid edges are true positives. From Eaton and Murphy (2007); Murphy (2023), Fig. 20.7.

Latent Space Interpolation and Arithmetic

One of the most interesting abilities of certain latent variable models is the ability to generate samples that have certain desired properties by interpolating between existing datapoints in latent space. Let x_1 and x_2 be two inputs (e.g., images), and let $z_1 = e(x_1)$ and $z_2 = e(x_2)$ be their latent encodings. We can regard z_1 and z_2 as two "anchors" in latent space. We can now generate new images that interpolate between these points by computing $z = \lambda z_1 + (1-\lambda)z_2$, where $0 \leq \lambda \leq 1$, and then decoding by computing $x' = d(z)$, where $d()$ is the decoder. This is called **latent space interpolation**, and will generate data that combines semantic features from both x_1 and x_2.

In some cases, we can go beyond interpolation, and can perform **latent space arithmetic**, in which we can increase or decrease the amount of a desired "semantic factor of variation". Let $\boldsymbol{z}_i^+$ and $\boldsymbol{z}_i^-$ be the average embeddings of images with and without attribute i. We define the offset vector as $\Delta_i = \boldsymbol{z}_i^+ - \boldsymbol{z}_i^-$. If we add some positive multiple of Δ_i to a new point z, we increase the amount of the attribute i; if we subtract some multiple of Δ_i, we decrease the amount of the attribute i.

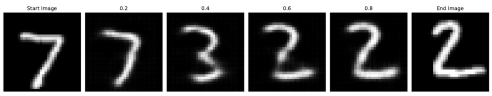


Figure 4: Interpolation between two MNIST images in the latent space of a β-VAE (with $\beta = 0.5$). Murphy (2023), Fig. 20.8.

The j'th reconstruction is computed using $\hat{x}_j = d\big(z + s_j\Delta\big)$, where $z = e(x)$ is the encoding of the original image, and s_j is a scale factor. In the textbook's CelebA example with attribute "wearing sunglasses", $s_j > 0$ adds sunglasses to the face. When $s_j < 0$ we remove sunglasses; but this also has the side effect of making the face look younger and more female, possibly a result of dataset bias.

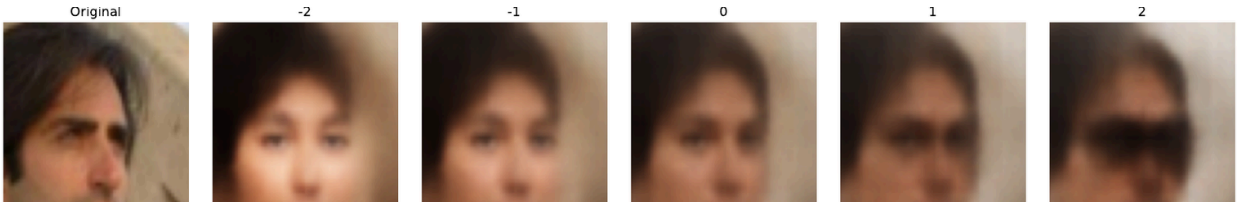


Figure 5: Arithmetic in the latent space of a β-VAE (with $\beta = 0.5$). The first column is an input image, with embedding z. Subsequent columns show the decoding of $z + s\Delta$, where $s \in \{-2, -1, 0, 1, 2\}$ and $\Delta = \boldsymbol{z}^+ - \boldsymbol{z}^-$ is the difference in the average embeddings of images with or without a certain attribute (here, wearing sunglasses). Murphy (2023), Fig. 20.10.

Discussion

Removing sunglasses with Δ also made the face look younger and more female. Stated in terms of the distribution over latents, what must be true of the training images for Δ to carry more than the sunglasses attribute?

III Evaluating Generative Models

Textbook: §20.4. Theis, van den Oord & Bethge (2016), A note on the evaluation of generative models.

Evaluating generative models requires metrics for

- *sample quality:* are samples generated by the model a part of the data distribution?
- *sample diversity:* are samples from the model distribution capturing all modes of the data distribution?
- *generalization:* is the model generalizing beyond the training data?

There is no known metric which meets all these requirements, but various metrics have been proposed to capture different aspects of the learned distribution, some of which we discuss below.

Likelihood-Based Evaluation

A standard way to measure how close a model q is to a true distribution p is in terms of the KL divergence:

$$D_{\mathrm{KL}}(p \parallel q) = \int p(x) \log \frac{p(x)}{q(x)} \, \mathrm{d}x = -H(p) + H_{\mathrm{ce}}(p, q), \tag{3}$$

where $H(p)$ is a constant, and $H_{\mathrm{ce}}(p, q)$ is the cross entropy. If we approximate $p(x)$ by the empirical distribution, we can evaluate the cross entropy in terms of the empirical negative log likelihood on the dataset:

$$\mathrm{NLL} = -\frac{1}{N} \sum_{n=1}^{N} \log q(x_n). \tag{4}$$

Usually we care about negative log likelihood on a held-out test set.

Unfortunately, for many models, computing the likelihood requires knowing the normalization constant of the probability model. Variational inference gives lower (and sometimes upper) bounds on the log likelihood, and annealed importance sampling estimates it by Monte Carlo. For implicit generative models, such as GANs, the likelihood is not even defined, so we need metrics that do not rely on likelihood.

A more subtle concern with likelihood is that it is often uncorrelated with the perceptual quality of the samples, at least for real-valued data, such as images and sound. In particular, a model can have great log-likelihood but create poor samples and vice versa. To see why a model can have good likelihoods but create bad samples, consider the following argument from Theis et al. Suppose q_0 is a density model for D-dimensional data x which performs arbitrarily well as judged by average log-likelihood, and suppose q_1 is a bad model, such as white noise. Consider the mixture model

$$q_2(x) = 0.01 q_0(x) + 0.99 q_1(x). \tag{5}$$

Clearly 99% of the samples will be poor. However, the log-likelihood per pixel will hardly change between q_2 and q_0 if D is large, since

$$\begin{aligned}\log q_2(x) &= \log[0.01q_0(x) + 0.99q_1(x)] \\ &\geq \log[0.01q_0(x)] = \log q_0(x) - 2. \end{aligned} \tag{6}$$

For high-dimensional data, $|\log q_0(x)| \sim D \gg 100$, so $\log q_2(x) \approx \log q_0(x)$, and hence mixing in the poor sampler does not significantly impact the log likelihood.

Now consider a case where the model has good samples but bad likelihoods. To achieve this, suppose q is a GMM centered on the training images:

$$q(x) = \frac{1}{N}\sum_{n=1}^{N} \mathcal{N}(x|x_n, \varepsilon^2 I). \tag{7}$$

If ε is small enough that the Gaussian noise is imperceptible, samples from this model will look good, since they correspond to the training set. But the model will almost certainly have poor likelihood on the test set due to overfitting: it has effectively just memorized the training set.

Distances and Divergences in Feature Space

Due to the challenges of comparing distributions in high dimensional spaces, most metrics used to evaluate generative models do not directly compare raw data (e.g., pixels) but use a neural network, commonly a pretrained classifier, to obtain features, and compare the feature distribution of model samples with that of the dataset.

The **Inception score** measures the average KL divergence between the marginal distribution of class labels obtained from the samples $p_\theta(y) = \int p_{\text{disc}}(y|x)p_\theta(x)\,\mathrm{d}x$ (where the integral is approximated by sampling images x from a fixed dataset) and the distribution $p(y|x)$ induced by samples from the model, $x \sim p_\theta(x)$. This leads to the following score:

$$\text{IS} = \exp\Big(\mathbb{E}_{p_\theta(x)}[D_{\text{KL}}(p_{\text{disc}}(Y|x) \parallel p_\theta(Y))]\Big). \tag{8}$$

To understand this, let us rewrite the log score as follows:

$$\log(\text{IS}) = H(p_\theta(Y)) - \mathbb{E}_{p_\theta(x)}[H(p_{\text{disc}}(Y|x))]. \tag{9}$$

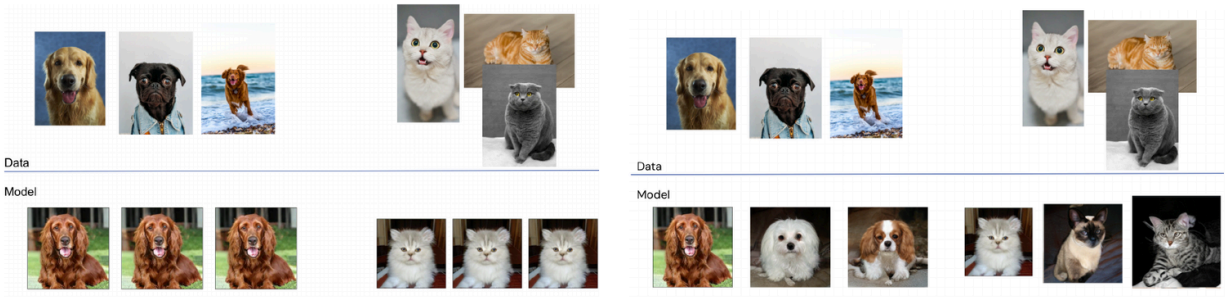
Thus we see that a high scoring model will be equally likely to generate samples from all classes, thus maximizing the entropy of $p_\theta(Y)$, while also ensuring that each individual sample is easy to classify, thus minimizing the entropy of $p_{\text{disc}}(Y|x)$.

The Inception score solely relies on class labels, and thus does not measure overfitting or sample diversity outside the predefined dataset classes. For example, a model which generates one perfect example per class would get a perfect Inception score,

despite not capturing the variety of examples inside a class. To address this drawback, the **Fréchet Inception distance** or FID score measures the Fréchet distance between two Gaussian distributions on sets of features of a pre-trained classifier. If we assume that the mean and covariance obtained from model features are μ_m and Σ_m and those from the data are μ_d and Σ_d, then the FID is

$$\text{FID} = \|\mu_m - \mu_d\|_2^2 + \text{tr}\Big(\Sigma_d + \Sigma_m - 2(\Sigma_d\Sigma_m)^{1/2}\Big). \tag{10}$$

Since it uses features instead of class logits, the Fréchet distance captures more than modes captured by class labels. Unlike the Inception score, a lower score is better since we want the two distributions to be as close as possible.



(a) *(b)*

Figure 6: (a) Model samples with good (high) inception score are visually realistic. (b) Model samples with good (low) FID score are visually realistic and diverse. Murphy (2023), Fig. 20.11.

Precision, Recall, and Other Checks

Since the FID only measures the distance between the data and model distributions, it is difficult to use it as a diagnostic tool: a bad (high) FID can indicate that the model is not able to generate high quality data, or that it puts too much mass around the data distribution, or that the model only captures a subset of the data. Trying to disentangle these failure modes has motivated individual **precision** (sample quality) and **recall** (sample diversity) metrics, computed from nearest neighbors in the feature space of a pretrained classifier. Precision measures whether samples are as close to data as data is to other data examples, while recall measures whether data is as close to model samples as model samples are to other samples.

Other checks include two sample tests (expensive, so unsuited to monitoring training), inspecting each sample's nearest neighbors in the dataset to test for memorization, and human evaluation, which suits art and other data for human display but is hard to standardize and expensive.

Discussion

A (probably too) common use of generative models is in the construction of synthetic data for other machine learning tasks. If the goal of a generative model is to produce synthetic data for a downstream classification model to train on, which metric from this section best captures what should be optimized for.